A GRAMMAR DEFINITION AND INSTANTIATION DETAILS

A.1 EQUIVALENCE RELATION DEFINITION

Precisely, $p_1 \sim p_2$ if and only if at least one of the following hold: $p_1 = p_2 = B_0$; or $p_i = B'_0(d_i, s_i)$ with $d_1 = d_2$; or $p_i = B_1(d_i, s_i, f_i)$ with $d_1 = d_2$; or $p_i = B_2^{(n)}(X_{1,i}, \ldots, X_{n,i}, c_i)$ with $X_{j,1} \sim X_{j,2}\ \forall 1 \leq j \leq n$; or $p_i = B_3^{(n)}(d_i, j_{1,i}, \ldots, j_{n,i}, s_1, s_2, c, f)$ where $d_1 = d_2$ and $j_{k,1} = j_{k,2}\ \forall 1 \leq k \leq n$. It is possible that generated formats equivalent in their string representation are not equivalent according to this equivalence relation.

A.1.1 VISUALIZATION OF PROMPT FORMAT'S PARSING AND FULL FORMAT GENERATION

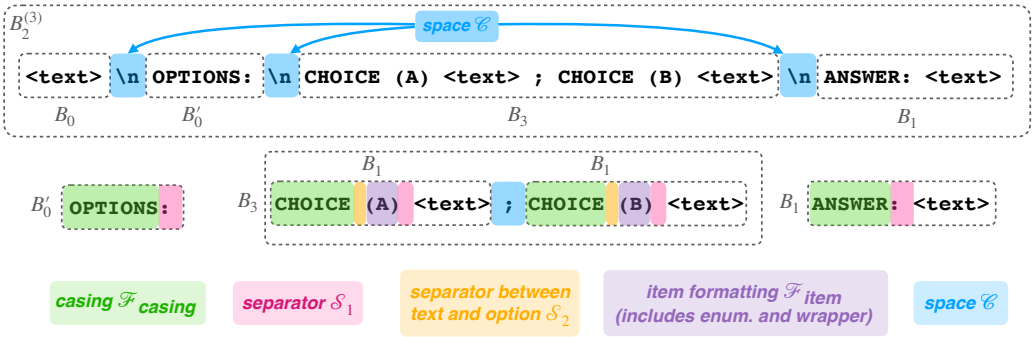

Figure 10: Visualization of a complex prompt format showing its parsing and which constants or functions affect each part of the format.

Figure 10 shows a visualization of how a complex format is parsed using our defined grammar. A full prompt consists of an instruction, n few-shots and a data point to solve. For example, if the instruction was `Given a sentence and two words that appear in it, answer which one of the two (A or B) appeared first in the sentence.`, a full prompt may look as follow. Note that we always use `\n\n` as space character between instruction and few-shots. The example below shows a 1-shot prompt. It is simply illustrative and does not correspond to any of the tasks considered.

```
Given a sentence and two words that appear in it, answer which one of
the two (A or B) appeared first in the sentence.

The quick brown fox jumps
OPTIONS:
CHOICE (A): fox ; CHOICE (B): brown
ANSWER: B

Over the lazy dog
OPTIONS:
CHOICE (A): lazy ; CHOICE (B): dog
ANSWER:
```

FORMATSPREAD forces all instantiations of a multiple choice variable to change jointly to maintain coherence, and this includes text in the instruction. Therefore, when changing the option items from A and B to I and II, the prompt will be generated as follows.

```
Given a sentence and two words that appear in it, answer which one of
the two (I or II) appeared first in the sentence.

The quick brown fox jumps
OPTIONS:
CHOICE (I): fox ; CHOICE (II): brown
ANSWER: II

Over the lazy dog
OPTIONS:
CHOICE (I): lazy ; CHOICE (II): dog
ANSWER:
```

A.2 ALLOWED VALUES FOR EACH SET $\mathcal{S}_1, \mathcal{S}_2, \mathcal{C}, \mathcal{F}_{\text{CASING}}, \mathcal{F}_{\text{ITEM}}$

$$
\begin{aligned}
\mathcal{S}_1 &= \{'','\ ','\backslash\mathtt{n}','\backslash\mathtt{n}','--','\ ','\ ';\ \backslash\mathtt{n}',\ ||\ ',' < \mathtt{sep} > ',' --',',',\ ',\ \backslash\mathtt{n}',\ ,',\backslash\mathtt{n}',.',\ ,\ '\} \\
\mathcal{S}_2 &= \{'','\ ','\ \ ',`\backslash\mathtt{t}'\} \text{ (no space, single space, double space, tab)} \\
\mathcal{C} &= \{'',':::\ ',':: \ ',': \ ',\ \backslash\mathtt{n}\backslash\mathtt{t}',\backslash\mathtt{n}\ \ ',\ :\ ',\ -',\ ',\backslash\mathtt{n}\ ',\backslash\mathtt{n}\backslash\mathtt{t}',:',::',-',\backslash\mathtt{t}'\} \\
\mathcal{F}_{\text{casing}} &= \{\mathtt{f(x)} = \mathtt{x}, \mathtt{f(x)} = \mathtt{x.title()}, \mathtt{f(x)} = \mathtt{x.upper()}, \mathtt{f(x)} = \mathtt{x.lower()}\} \\
\mathcal{F}_{\text{item}} &= \{x \mapsto f(g(x)) \mid \text{ such that } f \in \mathcal{F}_{\text{item1}} \wedge g \in \mathcal{F}_{\text{item2}}\} \\
\mathcal{F}_{\text{item1}} &= \{\mathtt{x} \mapsto (\mathtt{x}), \mathtt{x} \mapsto \mathtt{x.}, \mathtt{x} \mapsto \mathtt{x}), \mathtt{x} \mapsto \mathtt{x}\text{␣}), \mathtt{x} \mapsto [\mathtt{x}], \mathtt{x} \mapsto < \mathtt{x} >\} \\
\mathcal{F}_{\text{item2}} &= \{\mathtt{x} \to \mathtt{x} + 1, \mathtt{x} \to' \mathtt{A}' + \mathtt{x}, \mathtt{x} \to' \mathtt{a}' + \mathtt{x}, \\
&\qquad \mathtt{x} \to \mathtt{0x215F} + \mathtt{x} + 1, \mathtt{x} \to \mathtt{ROMAN[x].lower()}, \mathtt{x} \to \mathtt{ROMAN[x].upper()}\}
\end{aligned}
$$

Enumerations are indexed from (i.e., "1, 2, 3" rather than "0, 1, 2"). `ROMAN[x]` represents the Roman numerals written in regular ASCII characters. `'0x215F'+x` represent the series of Unicode characters for Roman numerals. ␣ denotes a spacing character for clarity.

A.3 RESTRICTIONS TO PROMPT FORMATS SPACES AND SEPARATORS' COMBINATIONS

We define several restrictions to ensure format naturalness. Users can additionally customize FORMATSPREAD by defining their own rules and restrictions between values. Our rules are as follows:

- If $B_2(X_1,\ldots,X_n,c)$ where c does not contain a newline, then each X_i's separators and any subcomponents' separators should not contain a newline.
- Similar to the rule above, if $B_3^{(n)}(d, j_1, \ldots, j_n, s_1, s_2, c, f_1, f_2)$ such that some separator contains a newline (i.e. s_1 contains a newline and/or s_2 contains a newline) then the space c must also contain a newline.
- For $B_1(d, s, f) := f(d)s< \mathtt{text} >$, s must not be the empty string (i.e., there has to be some separation between descriptor and text).
- Having c be an empty string space in $B_2^{(n)}$ is only allowed if the first $n-1$ components are B_1 fields with an empty `<text>`. Similarly, the newline restrictions mentioned above only apply if the `<text>` is not empty. This rarely happens in prompt formats, but there are formats such as `Question: <text> Options: A. <text> B. <text>` where the `Options:` do not have a corresponding field.

A.4 THOMPSON SAMPLING PRIORS

For the first exploration (i.e., finding the best-performing prompt format), we set an informative prior $\text{Beta}(\alpha, \beta) := \text{Beta}\left(\max\left(\frac{\beta \cdot x}{1-x}, 1.1\right), 5\right)$ for all arms p_i, where x is the original format's accuracy. Our goal is to set an informative prior where the expected value of the prior distribution is the original format accuracy x, since a priori it is the only information we have about performance.

This restricts the parameters as follows:

$$\mathbb{E}[\text{Beta}(\alpha, \beta)] = \frac{\alpha}{\alpha + \beta} = x$$
$$\alpha = \alpha \cdot x + \beta \cdot x$$
$$\alpha = \frac{\beta \cdot x}{1 - x}$$

Since β will modulate how confident is the prior, and we want to avoid the model being overconfident, we fix $\beta = 5$. Because we want to have an informative prior Beta(α, β) with a Gaussian-like PDF, we force $\alpha > 1$ and $\beta > 1$. In extreme cases, forcing $\alpha > 1$ might alter the expected value. The first exploration's priors are thus exactly Beta(α, β) with $\alpha = \max\left(\frac{\beta \cdot x}{1-x}, 1.1\right)$ and $\beta = 5$ for all arms p_i.

For the second exploration (i.e., finding the worst-performing prompt format), the model has access to the first explorations' counters $S_i^{(E/B)}$ and $N_i^{(E/B)}$. Therefore, we set the second exploration's priors to be Beta $\left(\alpha + S_i^{(E/B)}, \beta + \left(N_i^{(E/B)} - S_i^{(E/B)}\right)\right)$.

B Additional Experiments' Information and Plots

B.1 Task selection

We use a number of heuristics to filter Super-NaturalInstructions tasks to our set of 53 evaluation tasks. Datasets should have at least 1000 samples to be considered. We also remove tasks whose instructions are too long (over 3,000 characters) and datasets with inputs longer than 2,000 characters, given that this makes performing inference at scale intractable. We also filter datasets whose valid outputs include more than 20 different strings, given that we focus on classification tasks.

We also removed tasks where we found a priori performance on the task was 0% accuracy using LLaMA-2-7B 1-shot. Some Super-NaturalInstructions tasks are derived from the same original dataset, but ask different questions. We did not include more than 4 tasks from the same original dataset.

Finally, we also searched for having socially impactful tasks. Those tasks were the only Super-NaturalInstructions tasks where we included a format if one was not provided by the dataset.

The selected tasks were the following 53: `task050`, `task065`, `task069`, `task070`, `task114`, `task133`, `task155`, `task158`, `task161`, `task162`, `task163`, `task190`, `task213`, `task214`, `task220`, `task279`, `task280`, `task286`, `task296`, `task297`, `task316`, `task317`, `task319`, `task320`, `task322`, `task323`, `task325`, `task326`, `task327`, `task328`, `task335`, `task337`, `task385`, `task580`, `task607`, `task608`, `task609`, `task904`, `task905`, `task1186`, `task1283`, `task1284`, `task1297`, `task1347`, `task1387`, `task1419`, `task1420`, `task1421`, `task1423`, `task1502`, `task1612`, `task1678`, `task1724`.

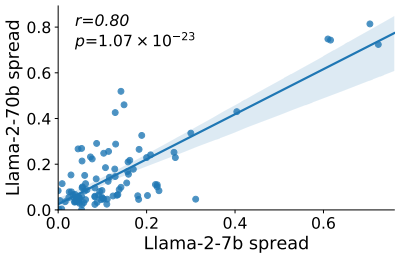

Figure 11: Comparison between Llama-2-7B and Llama-2-70B spreads. Llama-2-70B was computed using 4bit quantization (Dettmers et al., 2022).